

Networking Considerations

This report outlines the design considerations for transitioning the KIVI game project to a networked environment, addressing information and control flow between multiple computers.

1. Information Passing Between Computers

- Current State: The game state (board, players, dice) is encapsulated in the serializable GameManager class, enabling potential network transmission.
- Proposed Approach:
 - State Synchronization: Serialize GameManager instances (or deltas) to JSON/bytes for transmission. A central server maintains the "source of truth" to prevent desynchronization.
 - Efficiency: Transmit incremental updates (e.g., tile placements, rerolls) rather than full state snapshots to minimize bandwidth.

2. Control Passing Between Computers

- Current State: Local GameManager.currentPlayer() manages turn order.
- Proposed Approach:
 - Server-Managed Turns: A server arbitrates turn order, notifying clients when their turn begins. Clients send actions (e.g., place tile) to the server for validation before state updates.
 - UI Adjustments: Disable input during others' turns and display "Waiting for Player X" statuses.

3. Game Setup for Networking

- UI Modifications:
 - Hosting: Extend GameSettingScreen to include network options (max players, visibility).
 - Joining: Add a connection screen for IP input and lobby waiting.
- Lobby System: Hosts configure settings; joiners await game start.

4. Game State Storage and Transfer

- **Serialization:** Leverage existing GameManager serialization logic for network transmission.
- **Partial Updates:** Prioritize transmitting only changed data (e.g., updated tiles) to optimize performance.

5. Turn Control in a Networked Environment

- **Server Authority:** The server validates actions (e.g., tile validity) to prevent cheating.
- **Asynchronous Handling:** Clients request actions via the server, ensuring synchronized state transitions.

6. Starting/Ending a Networked Game

- **Host Initiation:** The host starts the game post-lobby, distributing the initial GameManager state.
- **Game Termination:** The server detects end conditions (e.g., 10 turns) and triggers WinnerGUI across all clients.

Challenges & Mitigations

- **Latency:** Implement client-side prediction for real-time feedback (e.g., dice rolls).
- **Concurrency:** Use thread-safe data structures on the server to manage simultaneous client requests.
- **Disconnections:** Introduce reconnection protocols and save-state recovery.

Conclusion

The project's use of a serializable GameManager and decoupled UI components provides a foundation for networking. Key adjustments include adopting a server-client model, modifying UI for network interactions, and ensuring robust state synchronization. Future work should prioritize latency reduction and concurrency management to deliver a seamless multiplayer experience.